

Problem Set 5 (Graphs - 1)

BM2043 - Algorithms and Data Structure Lab

October 6, 2024

Introduction to Graphs

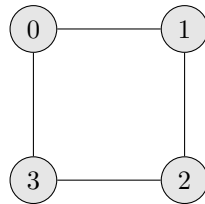
Graphs are fundamental data structures used to model pairwise relations between objects. A graph G consists of a set of vertices (also called nodes) V and a set of edges E that connect pairs of vertices. Graphs are widely used to represent networks, such as social networks, communication networks, and transportation networks.

Types of Graphs

Graphs can be classified based on the nature of their edges and the directionality of connections.

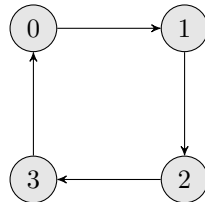
Undirected Graphs

In an **undirected graph**, the edges have no direction. That is, if there is an edge between vertex u and vertex v , you can traverse from u to v and from v to u . The edge is represented as an unordered pair (u, v) .



Directed Graphs (Digraphs)

In a **directed graph**, or **digraph**, edges have a direction associated with them. An edge from vertex u to vertex v means you can traverse from u to v , but not necessarily from v to u . The edge is represented as an ordered pair (u, v) .



Weighted and Unweighted Graphs

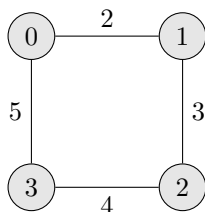
Graphs can also be classified based on whether their edges have weights.

Unweighted Graphs

In an **unweighted graph**, all edges are considered to have the same weight or no weight at all. The focus is on the connections between vertices rather than the cost associated with traversing an edge.

Weighted Graphs

In a **weighted graph**, each edge has an associated numerical value called a weight or cost. Weights can represent distances, costs, capacities, or any other quantifiable metric. Weighted graphs are useful for problems that involve finding the shortest path, minimal spanning tree, etc.



Graph Representations

Efficient graph algorithms depend on how the graph is represented in memory. The two most common representations are the adjacency list and the adjacency matrix.

Adjacency List

An **adjacency list** represents the graph as an array of lists. The index of the array represents a vertex, and each element in its list represents the vertices adjacent to it. This representation is space-efficient for sparse graphs (graphs with fewer edges).

- For vertex 0: adjacent vertices are [1, 2]
- For vertex 1: adjacent vertices are [0, 3]
- For vertex 2: adjacent vertices are [0, 3]
- For vertex 3: adjacent vertices are [1, 2]

Adjacency Matrix

An **adjacency matrix** is a $V \times V$ matrix A , where V is the number of vertices. The element $A[i][j]$ is 1 if there is an edge from vertex i to vertex j , and 0 otherwise. This representation is useful for dense graphs (graphs with many edges), and it allows quick edge existence checks.

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Applications of Graphs

Graphs are versatile structures that can model real-world systems:

- **Social Networks:** Nodes represent people, and edges represent friendships or connections.
- **Transportation Networks:** Nodes represent locations, and edges represent routes with distances or travel times.
- **Communication Networks:** Nodes represent devices, and edges represent communication links.
- **Web Graphs:** Nodes represent web pages, and directed edges represent hyperlinks.

Graph Traversal Algorithms

Graph traversal algorithms systematically visit all the vertices and edges of a graph. Two fundamental traversal techniques are:

- **Depth-First Search (DFS)**: Explores as far as possible along each branch before backtracking.
- **Breadth-First Search (BFS)**: Explores all the neighbor vertices at the present depth prior to moving on to the vertices at the next depth level.

Understanding these traversal methods is crucial for solving complex problems like finding connected components, detecting cycles, and searching for paths.

Conclusion

This introduction provides a foundation for understanding graphs, their types, and representations. As we delve into traversal algorithms like DFS and BFS, it's important to grasp these basic concepts to implement and analyze graph algorithms effectively.

Traversal

BFS and DFS

Objective: Given an undirected connected graph with V vertices numbered from 0 to $V - 1$, the task is to implement both **Depth First Search (DFS)** and **Breadth First Search (BFS)** traversals starting from the 0th vertex. The graph is represented using an adjacency list where $\text{adj}[i]$ contains a list of vertices connected to vertex i . Visit nodes in the order they appear in the adjacency list.

Graph Representation: Graphs can be represented in two common ways: adjacency lists and adjacency matrices.

Adjacency List: In an adjacency list, each vertex has a list of other vertices it is directly connected to. It is particularly space-efficient for sparse graphs. For example, if we have the following graph:

$$V = 5, \quad E = 4$$

Edges: $(0, 1), (0, 2), (2, 4), (0, 3)$

The corresponding adjacency list is:

$$\text{adj} = [[1, 2, 3], [0], [0, 4], [0], [2]]$$

This means:

- Vertex 0 is connected to vertices 1, 2, and 3.
- Vertex 1 is connected to vertex 0.
- Vertex 2 is connected to vertices 0 and 4.
- Vertex 3 is connected to vertex 0.
- Vertex 4 is connected to vertex 2.

Adjacency Matrix: In an adjacency matrix, the graph is represented as a $V \times V$ matrix where the entry at $\text{matrix}[i][j]$ is 1 if there is an edge between vertex i and vertex j , and 0 otherwise. For the same graph:

$$\text{adjacency matrix} = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

This representation is more memory-intensive for sparse graphs but makes it easy to check if two vertices are connected in constant time, i.e., $O(1)$.

Examples:

Example 1:

- **Input:**

$$V = 5, \quad \text{adj} = [[2, 3, 1], [0], [0, 4], [0], [2]]$$

- **Output:**

- DFS: [0, 2, 4, 3, 1]
- BFS: [0, 2, 3, 1, 4]

- **Explanation:**

- **DFS:** Start from vertex 0, visit vertex 2, then 4, backtrack to 0, visit 3, and finally 1. The traversal is $0 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$.
- **BFS:** Start from vertex 0, visit vertices 2, 3, and 1 (in the order they appear in the adjacency list), then visit 4 from vertex 2. The traversal is $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 4$.

Example 2:

- **Input:**

$$V = 4, \quad \text{adj} = [[1, 3], [2, 0], [1], [0]]$$

- **Output:**

- DFS: [0, 1, 2, 3]
- BFS: [0, 1, 3, 2]

- **Explanation:**

- **DFS:** Start from vertex 0, visit 1, then 2, backtrack to 0, and then visit 3. The traversal is $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$.
- **BFS:** Start from vertex 0, visit vertices 1 and 3, then visit 2 from vertex 1. The traversal is $0 \rightarrow 1 \rightarrow 3 \rightarrow 2$.

Constraints:

- $1 \leq V, E \leq 10^4$

Class Definition:

```
class Solution {
public:
    vector<int> dfsOfGraph(int V, vector<int> adj[]) {
        // Implement DFS traversal starting from vertex 0
    }

    vector<int> bfsOfGraph(int V, vector<int> adj[]) {
        // Implement BFS traversal starting from vertex 0
    }
};
```

DFS Traversal Hint: DFS explores as far as possible along each branch before backtracking, using either an explicit or implicit stack (recursion).

BFS Traversal Hint: BFS explores all the neighbors of a node before moving to the next level, using a queue data structure.

1 Traversal Problems

Problem 1: Number of Provinces

Objective: There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b , and city b is connected directly with city c , then city a is connected indirectly with city c .

A province is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an $n \times n$ matrix *isConnected* where *isConnected* $[i][j] = 1$ if the i th city and the j th city are directly connected, and *isConnected* $[i][j] = 0$ otherwise.

Return the total number of provinces.

Examples:

Example 1:

- **Input:**



isConnected =

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Output:**

2

Example 2:

- **Input:**



`isConnected =`

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Output:**

3

Constraints:

- $1 \leq n \leq 200$
- $n == isConnected.length$
- $n == isConnected[i].length$
- $isConnected[i][j]$ is 1 or 0.
- $isConnected[i][i] == 1$
- $isConnected[i][j] == isConnected[j][i]$

Class Definition:

```
class Solution {
public:
    int findCircleNum(vector<vector<int>>& isConnected) {
        // Implement the logic to find the number of provinces
    }
};
```

Problem 2: Flood Fill

Objective: You are given an image represented by an $m \times n$ grid of integers $image$, where $image[i][j]$ represents the pixel value of the image. You are also given three integers sr , sc , and $color$. Your task is to perform a flood fill on the image starting from the pixel $image[sr][sc]$.

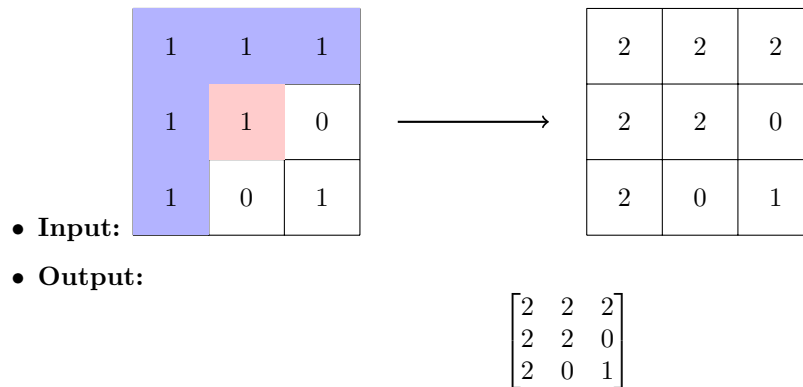
To perform a flood fill:

1. Begin with the starting pixel and change its color to $color$.
2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.
3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.
4. The process stops when there are no more adjacent pixels of the original color to update.

Return the modified image after performing the flood fill.

Examples:

Example 1:



Example 2:

- **Input:**

$$image = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad sr = 0, \quad sc = 0, \quad color = 0$$

- **Output:**

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Constraints:

- $m == image.length$
- $n == image[i].length$
- $1 \leq m, n \leq 50$
- $0 \leq image[i][j], color < 216$
- $0 \leq sr < m$
- $0 \leq sc < n$

0	0	0	0
1	0	1	0
0	1	1	0
0	0	0	0

Class Definition:

```
class Solution {
public:
    vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int color) {
        // Implement the logic to perform the flood fill
    }
};
```

Problem 3: Number of Enclaves

You are given an $m \times n$ binary matrix *grid*, where 0 represents a sea cell and 1 represents a land cell.

A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid.

Return the number of land cells in *grid* for which we cannot walk off the boundary of the grid in any number of moves.

Example 1

- **Input:**

```
grid = [[0,0,0,0],
        [1,0,1,0],
        [0,1,1,0],
        [0,0,0,0]]
```

- **Output:**

3

- **Explanation:** There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because it's on the boundary.

0	1	1	0
0	0	1	0
0	0	1	0
0	0	0	0

Example 2

• **Input:**

```
grid = [[0,1,1,0],
        [0,0,1,0],
        [0,0,1,0],
        [0,0,0,0]]
```

• **Output:**

0

- **Explanation:** All 1s are either on the boundary or can reach the boundary.

Constraints

- $m == \text{grid.length}$
- $n == \text{grid}[i].length$
- $1 \leq m, n \leq 500$
- $\text{grid}[i][j]$ is either 0 or 1.

Class Definition

```
class Solution {
public:
    int numEnclaves(vector<vector<int>>& grid) {
        // Implement the logic to calculate the number of enclaves
    }
};
```

Problem 4: Rotting Oranges

You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1 .

Example 1

- **Input:**

```
grid = [[2,1,1],
        [1,1,0],
        [0,1,1]]
```



(a) Minute 0



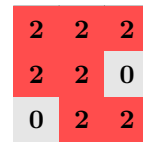
(b) Minute 1



(c) Minute 2



(d) Minute 3



(e) Minute 4

Figure 1: Progression of rotten oranges over time, with cell values

- **Output:**

4

Example 2

- **Input:**

```
grid = [[2,1,1],
        [0,1,1],
        [1,0,1]]
```

- **Output:**

-1

- **Explanation:** The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3

- **Input:**

```
grid = [[0,2]]
```

- **Output:**

0

- **Explanation:** Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints

- $m == \text{grid.length}$
- $n == \text{grid}[i].length$
- $1 \leq m, n \leq 10$
- $\text{grid}[i][j]$ is 0, 1, or 2.

Class Definition

```
class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        // Implement the logic to calculate the minimum time until all oranges are rotten
    }
};
```

Problem 5: Word Ladder

A transformation sequence from word **beginWord** to word **endWord** using a dictionary **wordList** is a sequence of words:

$$\text{beginWord} \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_k$$

such that:

- Every adjacent pair of words differs by a single letter.
- Every s_i for $1 \leq i \leq k$ is in **wordList**. Note that **beginWord** does not need to be in **wordList**.
- $s_k = \text{endWord}$

Given two words, **beginWord** and **endWord**, and a dictionary **wordList**, return the number of words in the shortest transformation sequence from **beginWord** to **endWord**, or 0 if no such sequence exists.

Example 1

- **Input:**

```
beginWord = "hit"
endWord = "cog"
wordList = ["hot","dot","dog","lot","log","cog"]
```

- **Output:**

- **Explanation:** One shortest transformation sequence is "hit" → "hot" → "dot" → "dog" → "cog", which is 5 words long.

Example 2

- **Input:**

```
beginWord = "hit"
endWord = "cog"
wordList = ["hot","dot","dog","lot","log"]
```

- **Output:**

0

- **Explanation:** The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.

Constraints

- $1 \leq \text{beginWord.length} \leq 10$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 5000$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are unique.

Class Definition

```
class Solution {
public:
    int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
        // Implement the logic to find the shortest transformation sequence length
    }
};
```

Problem 6: Number of Distinct Islands

Given a boolean 2D matrix `grid` of size $N \times M$, find the number of distinct islands where a group of connected 1s (horizontally or vertically) forms an island. Two islands are considered to be distinct if and only if one island is equal to another (not rotated or reflected).

Example 1

- **Input:**

1	1	0	0	0
1	1	0	0	0
0	0	0	1	1
0	0	0	1	1

`grid = [[1, 1, 0, 0, 0], [1, 1, 0, 0, 0], [0, 0, 0, 1, 1], [0, 0, 0, 1, 1]]`

- **Output:**

1

- **Explanation:** The same colored islands are equal. We have 2 equal islands, so we have only 1 distinct island.

Example 2

- **Input:**

1	1	0	1	1
1	0	0	0	0
0	0	0	0	1
1	1	0	1	1

`grid = [[1, 1, 0, 1, 1], [1, 0, 0, 0, 0], [0, 0, 0, 0, 1], [1, 1, 0, 1, 1]]`

- **Output:**

3

- **Explanation:** The same colored islands are equal. We have 4 islands, but 2 of them are equal, so we have 3 distinct islands.

Constraints

- $1 \leq N, M \leq 500$
- `grid[i][j] == 0 or 1`

Class Definition

```
class Solution {
public:
    int countDistinctIslands(vector<vector<int>>& grid) {
        // Implement the logic to count distinct islands
    }
};
```

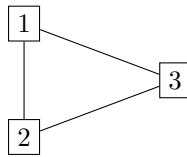
2 Graphs : Cycles

Problem 1: Cycle Detection

Objective: You are given an undirected graph with ***N*** vertices and ***M*** edges. The vertices are labeled from 1 to ***N***.

Your task is to determine whether the graph contains a cycle.

A cycle is defined as a path that starts from a given vertex and ends at the same vertex, traversing the edges only once.



Example: In the graph above, there exists a cycle involving vertices 1, 2, and 3.

Note:

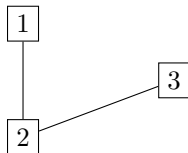
1. There are no parallel edges between two vertices.
2. There are no self-loops (an edge connecting a vertex to itself) in the graph.
3. The graph may be disconnected.

Input:

- ***N***: The number of vertices in the graph.
- ***Edges***: A list of edges connecting the vertices, represented as pairs.

Sample Input:

Input: $N = 3$, $Edges = [[1, 2], [2, 3], [1, 3]]$



Output:

Yes

Explanation: There are a total of 3 vertices in the graph, with edges between vertex 1 and 2, vertex 2 and 3, and vertex 1 and 3. Thus, a cycle exists in the graph.

Constraints:

- $1 \leq T \leq 10$
- $1 \leq N \leq 5000$
- $0 \leq M \leq \min(5000, (N \times (N - 1))/2)$
- $1 \leq \text{edges}[i][0] \leq N$
- $1 \leq \text{edges}[i][1] \leq N$

Time Limit: 1 second

Sample Input 1:

```
1
3 2
1 2
2 3
```

Sample Output 1:

No

Explanation of Sample Output 1: The above graph can be represented as follows:

With a total of 3 vertices, there is an edge between vertex 1 and 2, and an edge between vertex 2 and 3. Therefore, there is no cycle present in the graph.

Sample Input 2:

```
2
4 0
4 3
1 4
4 3
3 1
```

Sample Output 2:

No
Yes

Class Definition:

```
class Solution {
public:
    string cycleDetection(vector<vector<int>>& edges, int n, int m) {
        // (BFS Traversal) Write your code here.
    }
};
```

```

class Solution {
public:
    string cycleDetection(vector<vector<int>>& edges, int n, int m) {
        // (DFS Traversal) Write your code here.
    }
};

```

Problem 2: Is Graph Bipartite?

There is an undirected graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array **graph**, where **graph**[u] is an array of nodes that node u is adjacent to. The graph has the following properties:

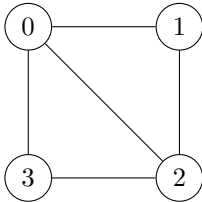
- There are no self-edges (**graph**[u] does not contain u).
- There are no parallel edges (**graph**[u] does not contain duplicate values).
- If v is in **graph**[u], then u is in **graph**[v] (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B .

Return **true** if and only if it is bipartite.

Example 1

- **Input:**



```
graph = [[1,2,3],[0,2],[0,1,3],[0,2]]
```

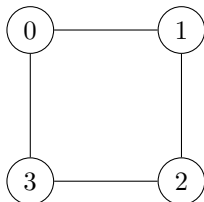
- **Output:**

```
false
```

- **Explanation:** There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.

Example 2

- **Input:**



```
graph = [[1,3],[0,2],[1,3],[0,2]]
```

- **Output:**

```
true
```

- **Explanation:** We can partition the nodes into two sets: $\{0, 2\}$ and $\{1, 3\}$.

Constraints

- `graph.length == n`
- $1 \leq n \leq 100$
- $0 \leq \text{graph}[u].\text{length} < n$
- $0 \leq \text{graph}[u][i] \leq n - 1$
- `graph[u]` does not contain u .
- All the values of `graph[u]` are unique.
- If `graph[u]` contains v , then `graph[v]` contains u .

Class Definition

```
class Solution {
public:
    bool isBipartite(vector<vector<int>>& graph) {
        // Implement the logic to check if the graph is bipartite
    }
};
```

Intuition A **bipartite graph** is a graph whose vertices can be divided into two disjoint sets A and B such that every edge connects a vertex in A to one in B . This is equivalent to being able to color the graph using two colors in such a way that no two adjacent vertices share the same color.

An important property of bipartite graphs is that they do not contain **odd-length cycles**. Any graph that contains a cycle of odd length cannot be bipartite because it's impossible to assign two colors without having two adjacent vertices sharing the same color.

To check if a graph is bipartite, we can attempt to color it using two colors while traversing the graph (e.g., using Depth-First Search or Breadth-First Search). If we encounter a situation where an adjacent vertex has the same color, the graph is not bipartite.

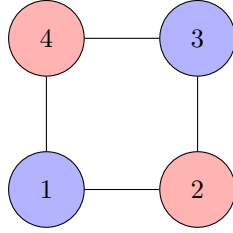


Figure 2: An even-length cycle (4 nodes) which is bipartite.

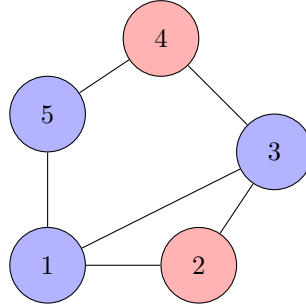


Figure 3: An odd-length cycle (5 nodes) which is not bipartite.

Examples In the first graph, we have an even-length cycle with 4 nodes. We can successfully color the graph using two colors (blue and red) such that no two adjacent vertices share the same color. This demonstrates that the graph is bipartite.

In the second graph, we have an odd-length cycle with 5 nodes. When we attempt to color the graph using two colors, we find that two adjacent vertices share the same color (nodes "1" and "3" are both blue and connected). This conflict shows that the graph is not bipartite.

Problem 3: Topological Sort or Kahn's Algorithm

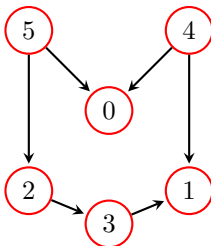
Problem Statement:

Given a Directed Acyclic Graph (DAG) with V vertices labeled from 0 to $V - 1$ and E edges. The graph is represented using an adjacency list where `adj[i]` lists all nodes connected to node i . Find any Topological Sorting of that graph.

In topological sorting, node u will always appear before node v if there is a directed edge from node u towards node v (i.e., $u \rightarrow v$).

Example 1

• Input:



$V = 6$, $\text{adj} = [[], [], [3], [1], [0, 1], [0, 2]]$

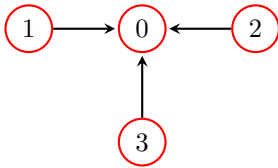
- **Output:**

[5, 4, 2, 3, 1, 0]

- **Explanation:** A graph may have multiple topological sortings. The result is one of them. Necessary conditions for the ordering are satisfied, e.g., according to edge $5 \rightarrow 0$, node 5 must appear before node 0.

Example 2

- **Input:**



$V = 4$, $\text{adj} = [[], [0], [0], [0]]$

- **Output:**

[3, 2, 1, 0]

- **Explanation:** Necessary conditions for the ordering are satisfied, e.g., for edge $1 \rightarrow 0$, node 1 must appear before node 0.

Constraints

- E = number of edges
- $1 \leq V, E \leq 10^4$

Class Definition

```
class Solution {
public:
    vector<int> topoSort(int V, vector<int> adj[]) {
        // Implement DFS topological sorting here
    }
};

class Solution {
public:
    vector<int> topoSort(int V, vector<int> adj[]) {
        // Implement BFS topological sorting here
    }
};
```

```

class Solution {
public:
    vector<int> topoSort(int V, vector<int> adj[]) {
        // Implement Kahn's algorithm for topological sorting here
    }
};

```

Problem 4: Course Schedule

Problem Statement:

You have a total of `numCourses` courses labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course *bi* first if you want to take course *ai*.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1

- **Input:**

`numCourses = 2, prerequisites = [[1,0]]`

- **Output:**

`[0,1]`

- **Explanation:** To take course 1, you should have finished course 0. So the correct course order is `[0,1]`.

Example 2

- **Input:**

`numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]`

- **Output:**

`[0,2,1,3]`

- **Explanation:** To take course 3, you should have finished both courses 1 and 2. A valid order could be `[0,1,2,3]` or `[0,2,1,3]`.

Example 3

- **Input:**

`numCourses = 1, prerequisites = []`

- **Output:**

`[0]`

Constraints

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq \text{numCourses} \times (\text{numCourses} - 1)$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- $a_i \neq b_i$
- All pairs $[a_i, b_i]$ are distinct.

Class Definition

```
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        // Implement Kahn's algorithm or DFS for topological sorting here
    }
};
```

Problem 5: Alien Dictionary

Problem Statement:

You are given a sorted dictionary of an alien language having N words and K starting alphabets of a standard dictionary. Your task is to find the order of characters in the alien language.

Example 1

- **Input:**

$N = 5, K = 4, \text{dict} = ["baa", "abcd", "abca", "cab", "cad"]$

- **Output:**

"bdac"

- **Explanation:** The order of characters can be determined by analyzing the consecutive pairs of words.

Example 2

- **Input:**

$N = 3, K = 3, \text{dict} = ["caa", "aaa", "aab"]$

- **Output:**

"cab"

- **Explanation:** Analyzing consecutive pairs reveals the character order.

Constraints

- $1 \leq N, M \leq 300$
- $1 \leq K \leq 26$
- $1 \leq \text{dict}[i].\text{length} \leq 50$

Class Definition

```
class Solution {
public:
    string findOrder(string dict[], int N, int K) {
        // Implement the logic to determine character order here
    }
};
```